

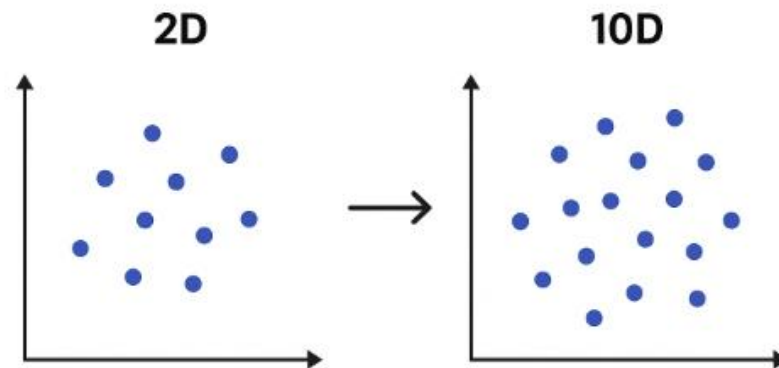
Intro to Dimensionality Reduction

Week # 11

Course Instructor: Mahrukh Khan

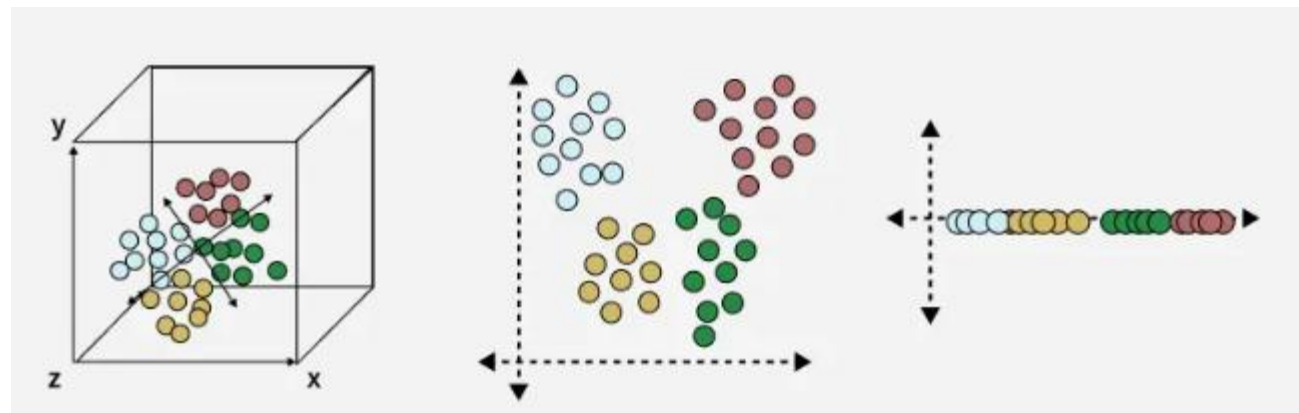
Curse of Dimensionality

- Too many features = slower learning + overfitting
- **Sparsity**->Data points are far apart → no dense clusters → distance metrics (like Euclidean) lose meaning.
- **Overfitting**->Models learn noise, not signal, because every data point can look “unique”.
- **Computation Cost**->Training becomes slow: more dimensions = more weights, more time.
- **Visualization Impossible**->Can't plot or intuitively understand 100+ dimensions.



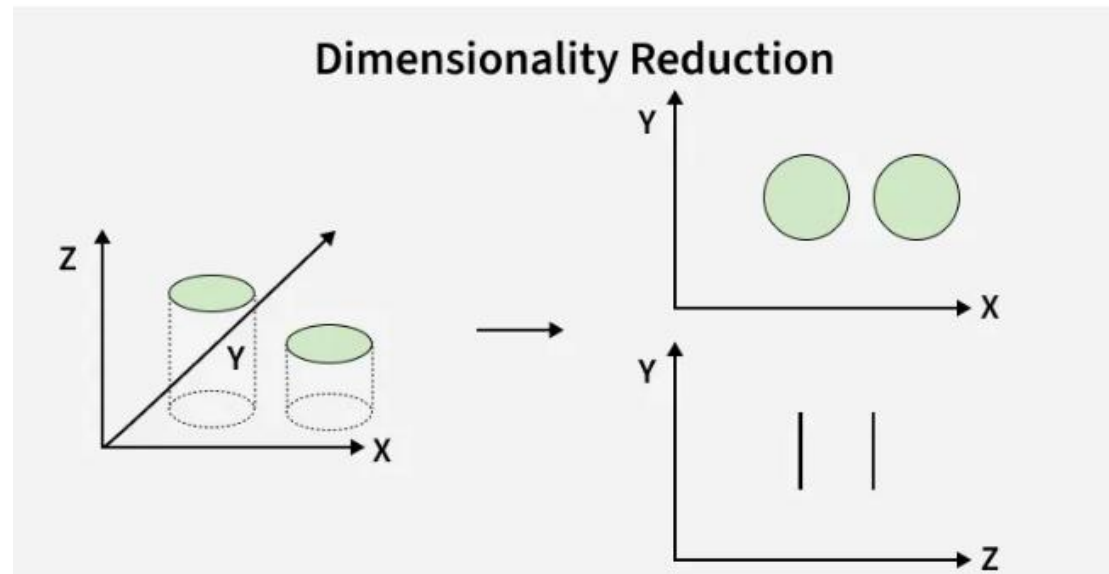
Dimensionality Reduction

- When working with machine learning models, datasets with too many features can cause issues like slow computation and overfitting. Dimensionality reduction helps to reduce the number of features while retaining key information. It converts high-dimensional data into a lower-dimensional space while preserving important details.



How Dimensionality Reduction Works?

- Let's understand how dimensionality Reduction is used with the help of example. Imagine a dataset where each data point exists in a 3D space defined by axes X, Y and Z. If most of the data variance occurs along X and Y then the Z-dimension may contribute very little to understanding the structure of the data.
- Before Reduction we can see that data exist in 3D (X,Y,Z). It has high redundancy and Z contributes little meaningful information
- On the right after reducing the dimensionality the data is represented in lower-dimensional spaces. The top plot (X-Y) maintains the meaningful structure while the bottom plot (Z-Y) shows that the Z-dimension contributed little useful information.
- This process makes data analysis more efficient hence improving computation speed and visualization while minimizing redundancy.



Advantages and Disadvantages of Dimensionality Reduction

Advantages

- ✓ Simplify the pattern representation and the classifiers
- ✓ Faster classifier with less memory consumption
- ✓ Alleviate curse of dimensionality with limited data sample



Disadvantages

- ✗ Loss information
- ✗ Increased error in the resulting recognition system

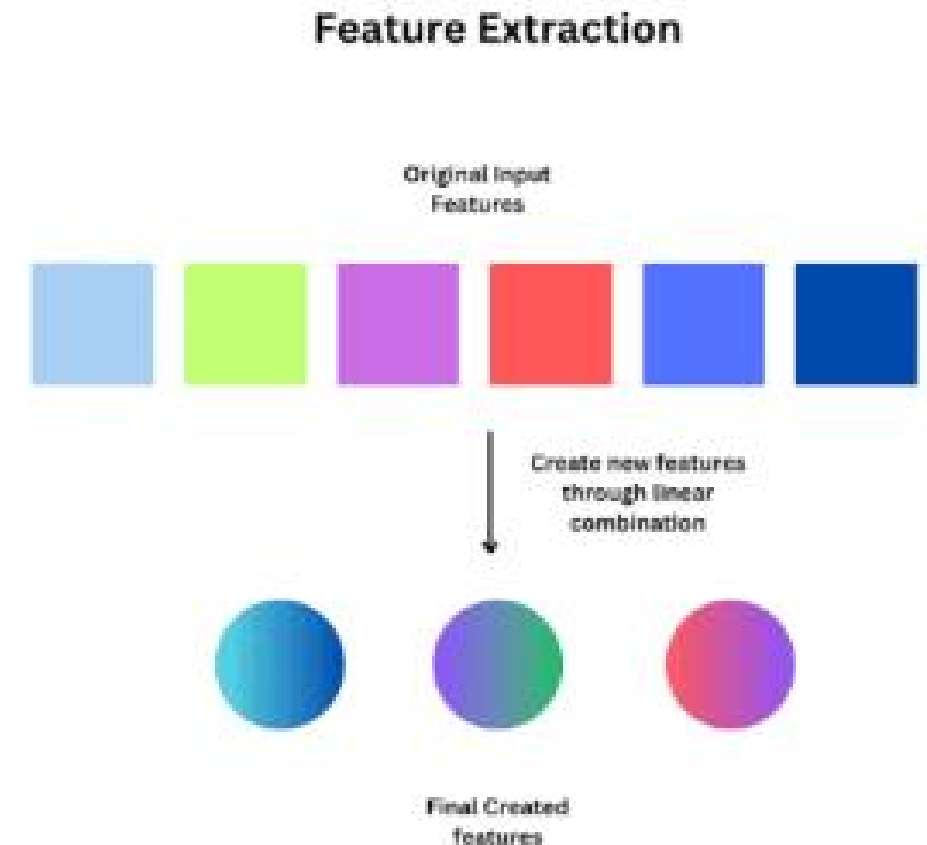


Feature Selection vs Feature Extraction

Concept	Description	Example
Feature Selection	Choose some existing features	Keep only “height” and “weight”
Feature Extraction	Create <i>new</i> features as combinations of old ones	Combine height & weight into a “body size” variable

What Is Feature Extraction?

- Feature extraction transforms raw data into meaningful features for analysis.
- Reduces complexity and highlights the most relevant information.
- Makes data easier for machine learning models to understand and process.

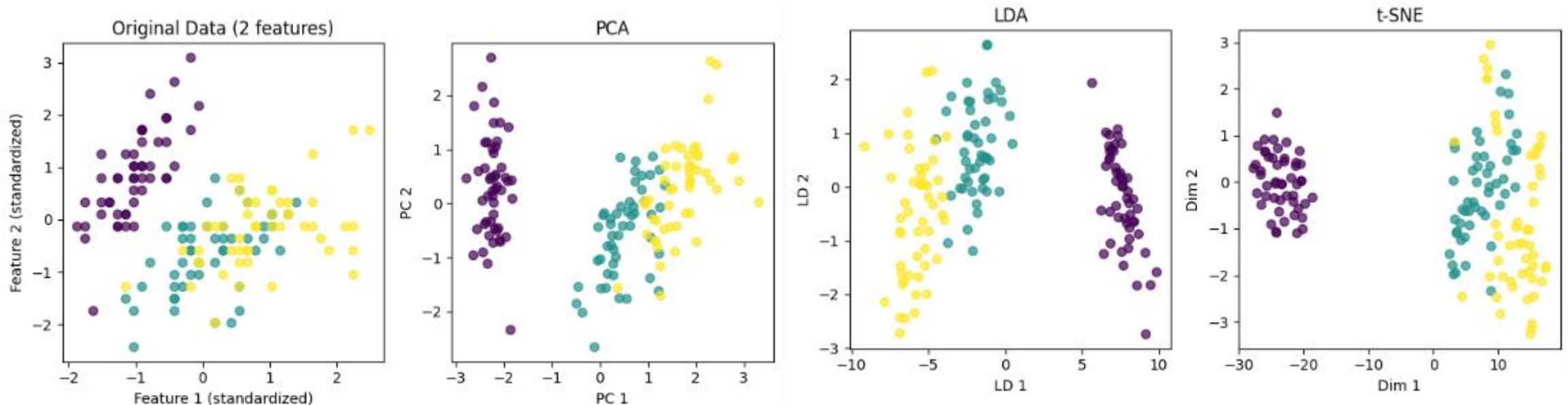


Feature Extraction Techniques

Data Type	Common Feature Extraction Techniques
Numerical	PCA, LDA, Statistical features
Text	TF-IDF, Word2Vec, BERT
Images	HOG, SIFT, CNN embeddings
Audio	MFCC, Spectral features
Time Series	FFT, Wavelet, Autocorrelation

Dimensionality Reduction Techniques

- **Principal Component Analysis**: It selects variables that account for most of the data's variation, simplifying the dataset by focusing on the most important components.
- **Linear Discriminant Analysis (LDA)**: It finds the best combination of features to separate different classes, maximizing class separability for better classification.
- **t-Distributed Stochastic Neighbor Embedding (t-SNE)**: A technique that reduces high-dimensional data into two or three dimensions ideal for visualizing complex datasets.
- **SVD**



Principle Component Analysis (PCA)

- PCA uses linear algebra to transform data into new features called principal components.
- It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix.
- PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.
- **Goal:** Find directions (axes) that capture maximum variance.

How Principal Component Analysis Works

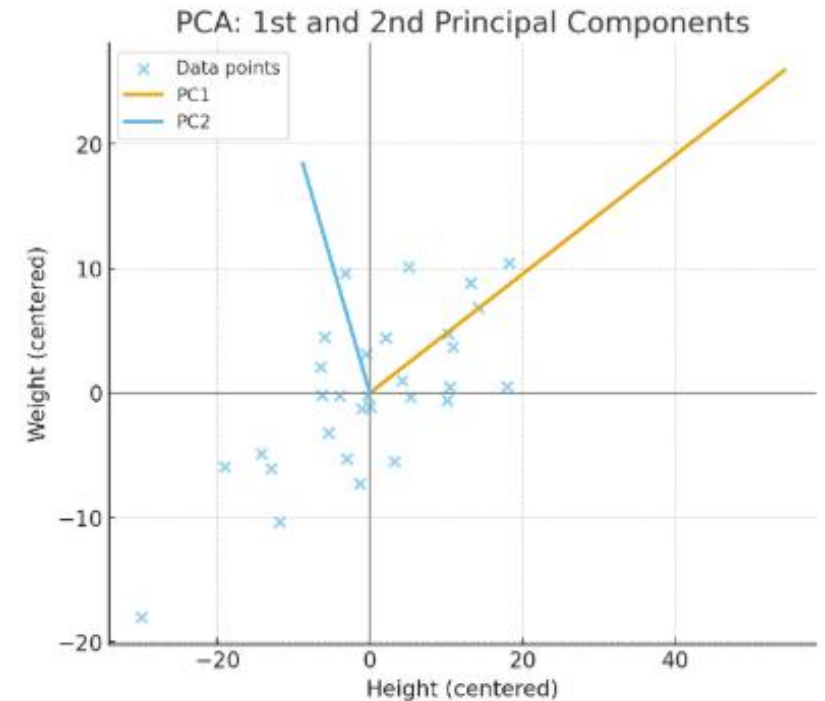
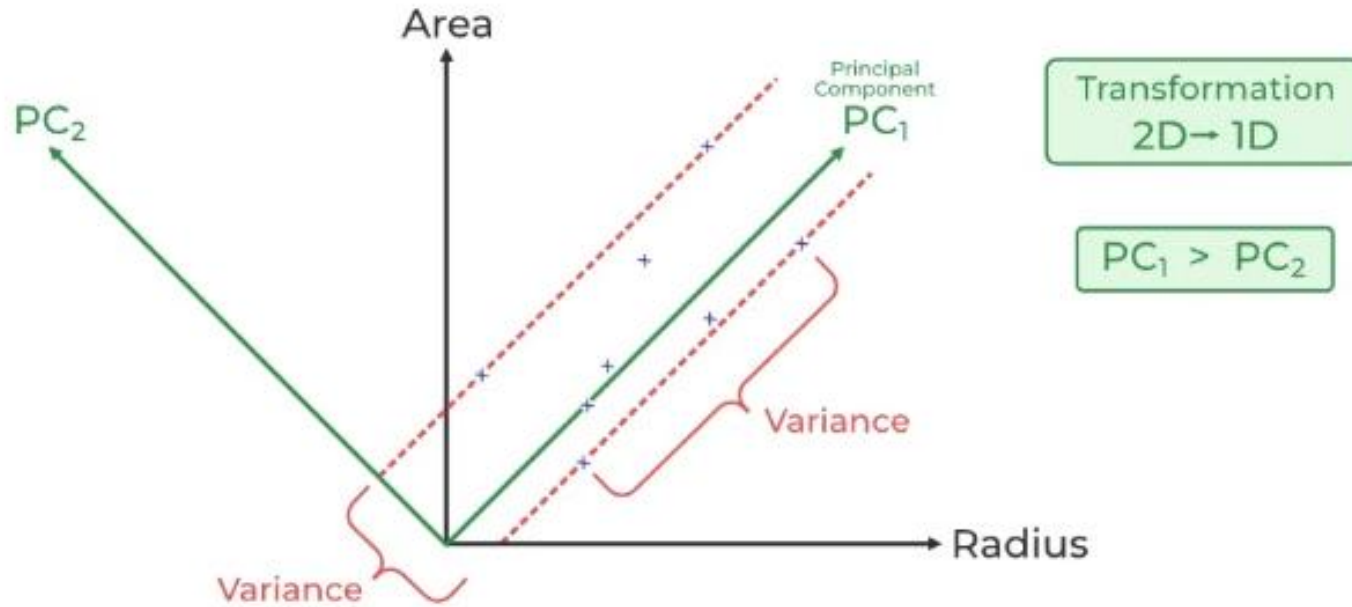
- **Step 1: Standardize the Data**->Different features may have different units and scales like salary vs. age. To compare them fairly PCA first standardizes the data by making each feature have:
 - A mean of 0
 - A standard deviation of 1
- **Step 2: Calculate Covariance Matrix** to see how features relate to each other whether they increase or decrease together. The covariance between two features x_1 and x_2 is:

$$cov(x_1, x_2) = \frac{\sum_{i=1}^n (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)}{n-1}$$

- **Step 3: Find the Principal Components**-> PCA identifies **new axes** where the data spreads out the most:
 - **1st Principal Component (PC1)**: The direction of maximum variance (most spread).
 - **2nd Principal Component (PC2)**: The next best direction, *perpendicular to PC1* and so on.
- **Step 4: Pick the Top Directions & Transform Data**
 - After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:
 - **Select the top k components** that capture most of the variance like 95%.
 - **Transform the original dataset** by projecting it onto these top components.

Here's the **2D scatter plot of Height vs. Weight**.

- The **blue dots** are centered data points.
- The **first principal component (PC1)** — the slanted line — shows the **direction of maximum variance (spread)** in the data.
- The **second component (PC2)** is **perpendicular** to it and captures the remaining variance.



- PCA = "rotation" of axes to summarize data.
- It is *unsupervised* — doesn't use labels (just structure).

Example: Student Academic Performance Dataset

Student	Height	Weight	Age	Study	Attend	T1	T2	T3	Final
1	160	55	18	10	80	65	70	72	75
2	165	58	18	12	85	70	74	76	80
3	170	60	19	15	90	78	82	85	88
4	172	65	19	16	92	80	84	88	91
5	175	68	20	18	95	85	87	90	94
6	180	75	21	20	97	90	92	94	96
7	178	72	21	19	94	88	91	93	95
8	160	50	18	5	60	40	50	55	58
9	162	52	18	6	65	45	55	60	62
10	168	59	19	14	88	75	78	82	85

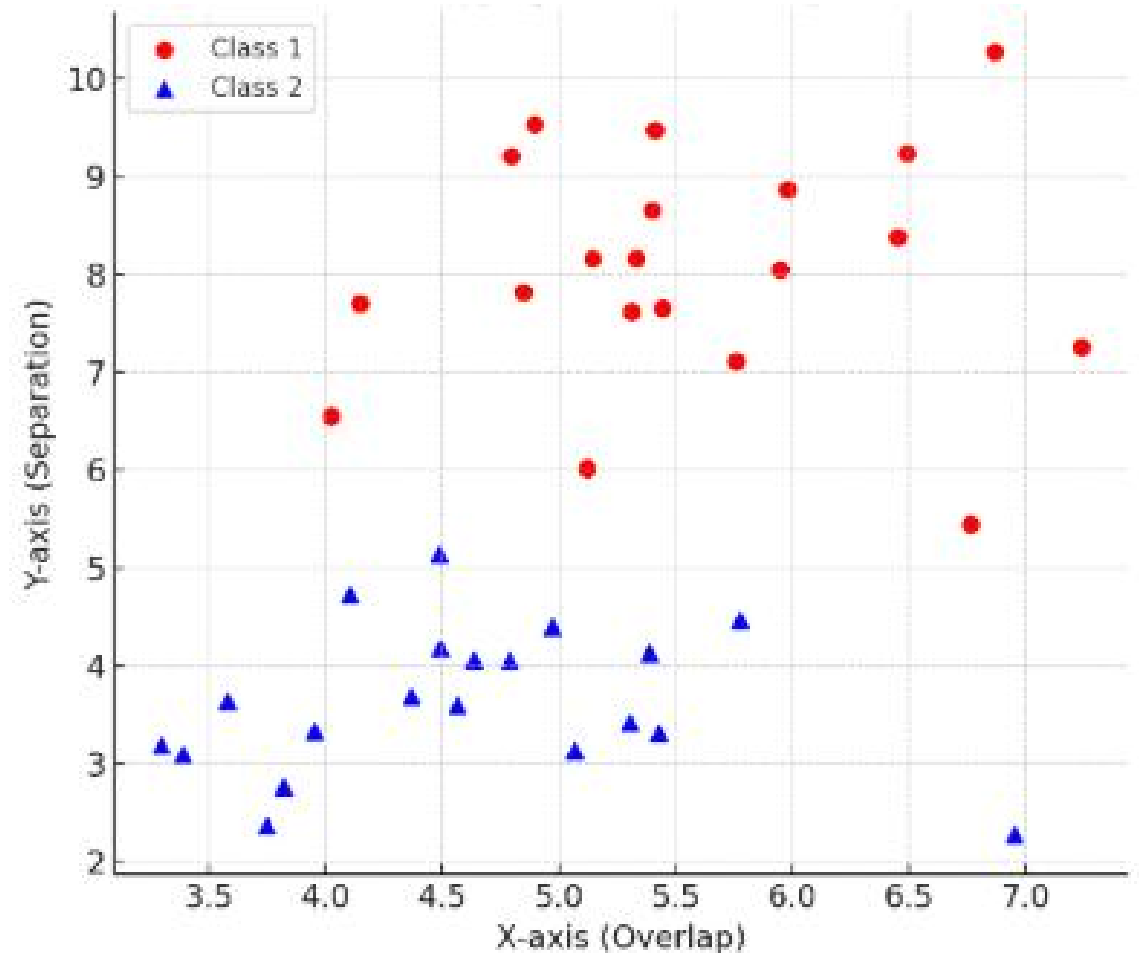
- We used PCA to reduce dimensions — but did PCA care about class labels?”
- “What if we *want* to find directions that best separate different classes?”

Intuition behind LDA — finding the projection that best separates classes.

Imagine each class as a **tight cloud** of points.

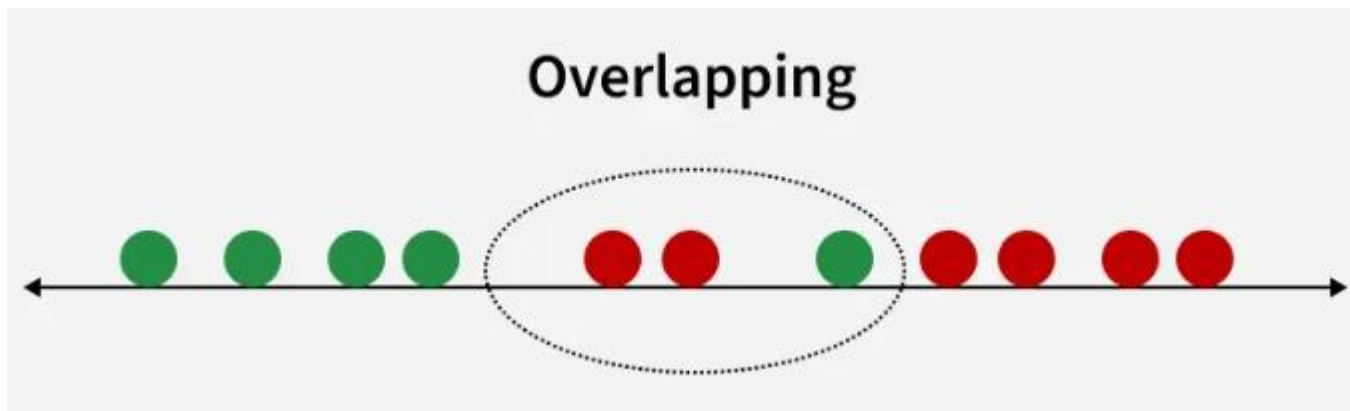
LDA finds a line such that:

- The **cloud centers (means)** are as far apart as possible.
- The **cloud spreads (variances)** are as small as possible along that line.



Linear Discriminant Analysis (LDA)

- Linear Discriminant Analysis (LDA) also known as Normal Discriminant Analysis helps separate two or more classes by converting higher-dimensional data space into a lower-dimensional space.
- It is used to identify a linear combination of features that best separates classes within a dataset.

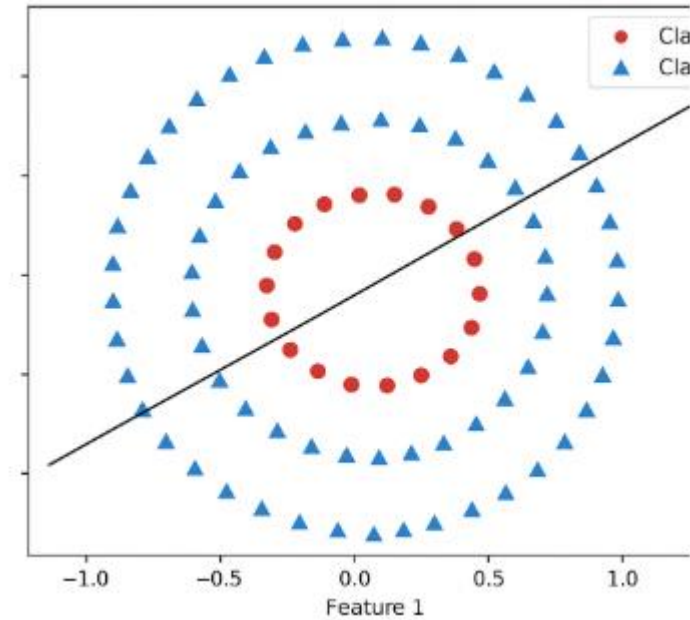


For example, we have two classes that need to be separated efficiently. Each class may have multiple features and using a single feature to classify them may result in overlapping.

How LDA work

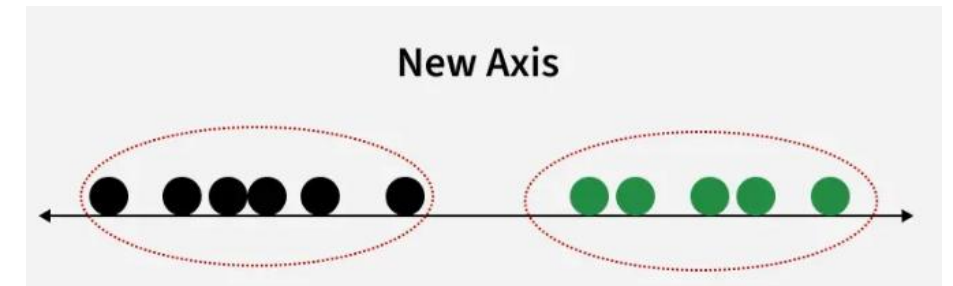
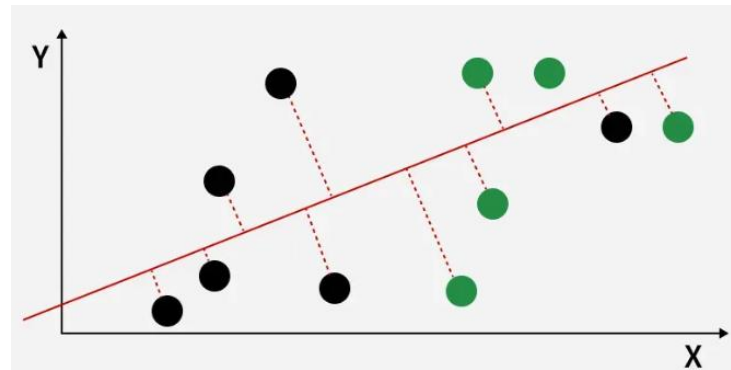
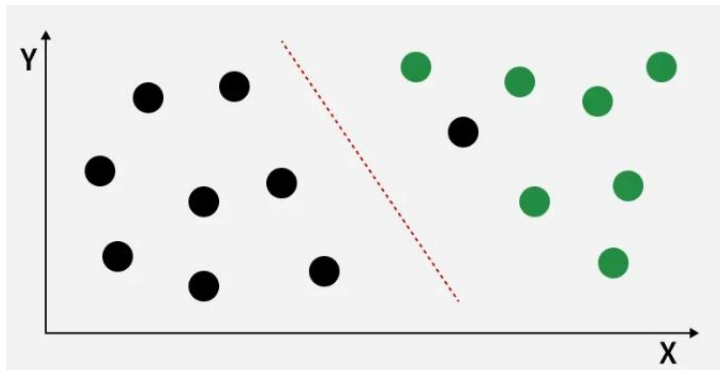
- LDA finds a **projection vector w** such that:
- $J(w) = \frac{\textit{Between-class variance}}{\textit{Within-class variance}}$
- That means we want large distance between class means and small scatter within each class.
- **Mathematically:** $J(w) = \frac{w^T S_B w}{w^T S_W w}$
 - where
 - S_B :between-class scatter matrix
 - S_W :within-class scatter matrix

When class distributions share the same mean, LDA cannot find a separating axis and non-linear discriminant analysis is needed



Assumptions of LDA

- For LDA to perform effectively, certain assumptions are made:
 - **Gaussian Distribution**: The data in each class should follow a normal bell-shaped distribution.
 - **Equal Covariance Matrices**: All classes should have the same covariance structure.
 - **Linear Separability**: The data should be separable using a straight line or plane.



PCA vs LDA

Aspect	PCA	LDA
Type	Unsupervised	Supervised
Focus	Maximize variance	Maximize class separability
Input	Features only	Features + Class labels
Uses	Noise reduction, visualization	Classification, pattern recognition
Basis	Eigen decomposition of covariance matrix	Eigen decomposition of scatter matrices

Example:

- **Compute class means:**
 - Class 0 mean $\mu_0 = [2.7, 3.04]$
 - Class 1 mean $\mu_1 = [6.1, 6.08]$
- **Compute between-class scatter:**
 - Difference of means $\rightarrow [3.4, 3.04]$
- **Compute within-class scatter:**

Each class is tight around its own mean.

 - Formally, for class i :
 - Then total **within-class scatter matrix**: $S_W = \sum_i S_i$
 - It's like summing up the covariance (without dividing by n) of each class.
- **Find discriminant direction w :**

LDA gives a line roughly along the vector $[3, 3]$, i.e. slope ≈ 1 .

Sample	Feature ₁ (x_1)	Feature ₂ (x_2)	Class
1	2.0	3.0	0
2	2.5	2.8	0
3	3.0	3.2	0
4	3.2	2.9	0
5	2.8	3.3	0
6	5.0	6.0	1
7	5.5	5.8	1
8	6.0	6.2	1
9	6.2	5.9	1
10	5.8	6.3	1

Computing So.

$$(x - \mu_i)(x - \mu_i)^T = \begin{bmatrix} (x_1 - \mu_{i1})^2 & (x_1 - \mu_{i1})(x_2 - \mu_{i2}) \\ (x_2 - \mu_{i2})(x_1 - \mu_{i1}) & (x_2 - \mu_{i2})^2 \end{bmatrix}$$

For each, compute $(x - \mu_0)(x - \mu_0)^T$

$$[-0.7, -0.04]^T [-0.7, -0.04] = \begin{bmatrix} (-0.7)^2 & (-0.7)(-0.04) \\ (-0.04)(-0.7) & (-0.04)^2 \end{bmatrix} = \begin{bmatrix} 0.49 & 0.028 \\ 0.028 & 0.0016 \end{bmatrix}$$

Point	(x - \mu_0)
[2.0, 3.0]	[-0.7, -0.04]
[2.5, 2.8]	[-0.2, -0.24]
[3.0, 3.2]	[0.3, 0.16]
[3.2, 2.9]	[0.5, -0.14]
[2.8, 3.3]	[0.1, 0.26]

$$\begin{bmatrix} 0.49 & 0.028 \\ 0.028 & 0.0016 \end{bmatrix}$$

$$\begin{bmatrix} 0.04 & 0.048 \\ 0.048 & 0.058 \end{bmatrix}$$

$$\begin{bmatrix} 0.09 & 0.048 \\ 0.048 & 0.026 \end{bmatrix}$$

$$\begin{bmatrix} 0.25 & -0.07 \\ -0.07 & 0.020 \end{bmatrix}$$

$$\begin{bmatrix} 0.01 & 0.026 \\ 0.026 & 0.068 \end{bmatrix}$$

$$S_0 = \begin{bmatrix} 0.49 + 0.04 + 0.09 + 0.25 + 0.01 & 0.028 + 0.048 + 0.048 - 0.07 + 0.026 \\ 0.028 + 0.048 + 0.048 - 0.07 + 0.026 & 0.0016 + 0.058 + 0.026 + 0.020 + 0.068 \end{bmatrix}$$

$$S_0 = \begin{bmatrix} 0.88 & 0.08 \\ 0.08 & 0.17 \end{bmatrix}$$

$$S_1 = \begin{bmatrix} 2.07 & 0.52 \\ 0.52 & 0.31 \end{bmatrix} \quad S_W = S_0 + S_1 = \begin{bmatrix} 2.95 & 0.60 \\ 0.60 & 0.48 \end{bmatrix}$$

$$S_W^{-1} = \frac{1}{1.056} \begin{bmatrix} 0.48 & -0.60 \\ -0.60 & 2.95 \end{bmatrix} = \begin{bmatrix} 0.4545 & -0.568 \\ -0.568 & 2.795 \end{bmatrix}$$

- To Find discriminant direction w :

$$\text{Compute } w = S_W^{-1}(\mu_1 - \mu_0)$$

$$w = \begin{bmatrix} 0.4545 & -0.568 \\ -0.568 & 2.795 \end{bmatrix} \begin{bmatrix} 3.4 \\ 3.04 \end{bmatrix} = \begin{bmatrix} (0.4545)(3.4) + (-0.568)(3.04) \\ (-0.568)(3.4) + (2.795)(3.04) \end{bmatrix} \quad w = \begin{bmatrix} -0.18 \\ 6.57 \end{bmatrix}$$

Compute the length (magnitude) of w

$$\|w\| = \sqrt{(-0.18)^2 + (6.57)^2}$$

To make w a **unit vector** (length = 1),
we divide each component of w by its length $\|w\|$:

$$w_{\text{norm}} = \begin{bmatrix} \frac{-0.18}{6.57} \\ \frac{6.57}{6.57} \end{bmatrix} = \begin{bmatrix} -0.027 \\ 0.999 \end{bmatrix}$$

let's now **project your data points** from both classes onto the **LDA discriminant axis**

Class	Mean Projection	Range
Class 0	≈ 2.97	2.73 – 3.22
Class 1	≈ 5.88	5.64 – 6.14